

تغرات RET Hook الحرجة — ANOGS/AnoSDK

تحليل معماري دفاعي: أي أوفست إذا تعرّض لـ RET يُعطي الحماية 100%

✦ bypass by Hdanh ✦ — ✦ made by @Nopermcc ✦

تقرير دفاعي — لا يحتوي على تعليمات تعطيل أو patch bytes

0

حماية ذاتية

6

نقطة اختناق SPOF

24

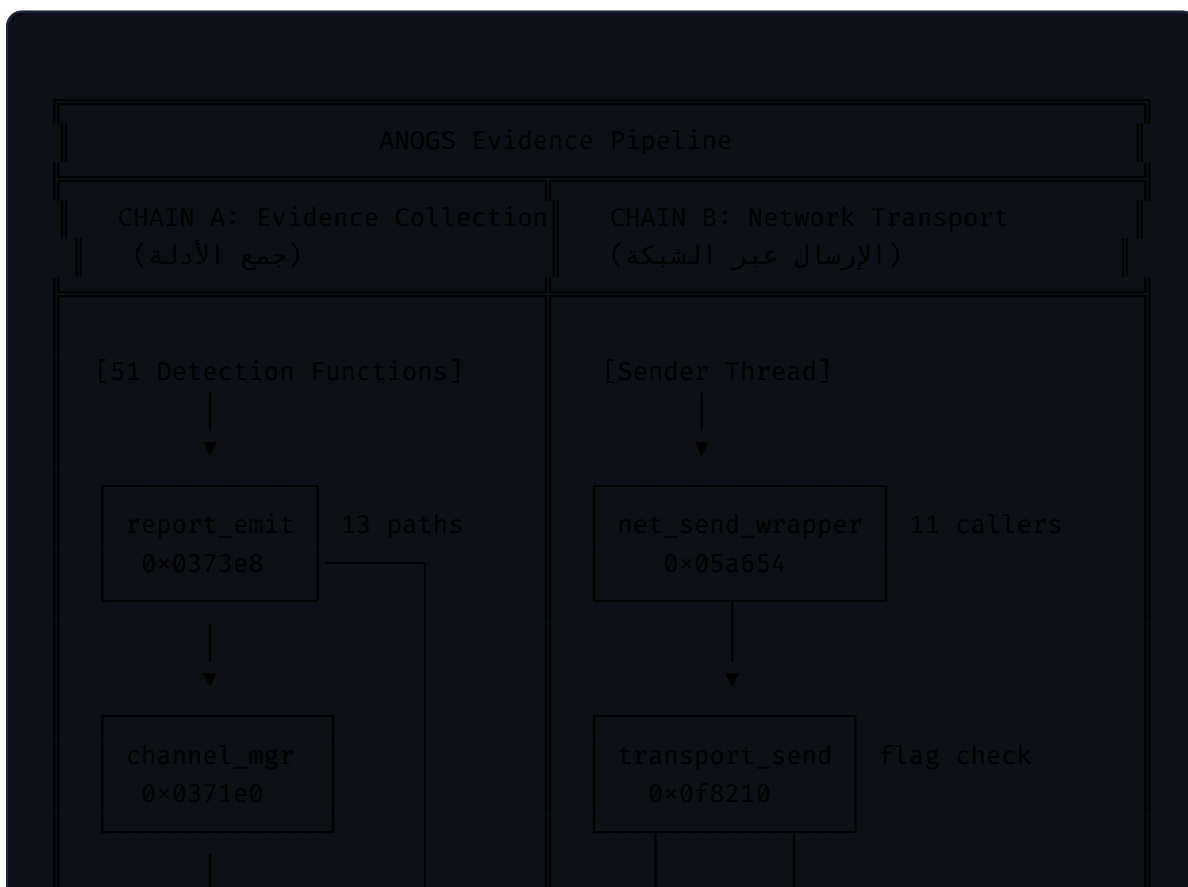
تصل للشبكة

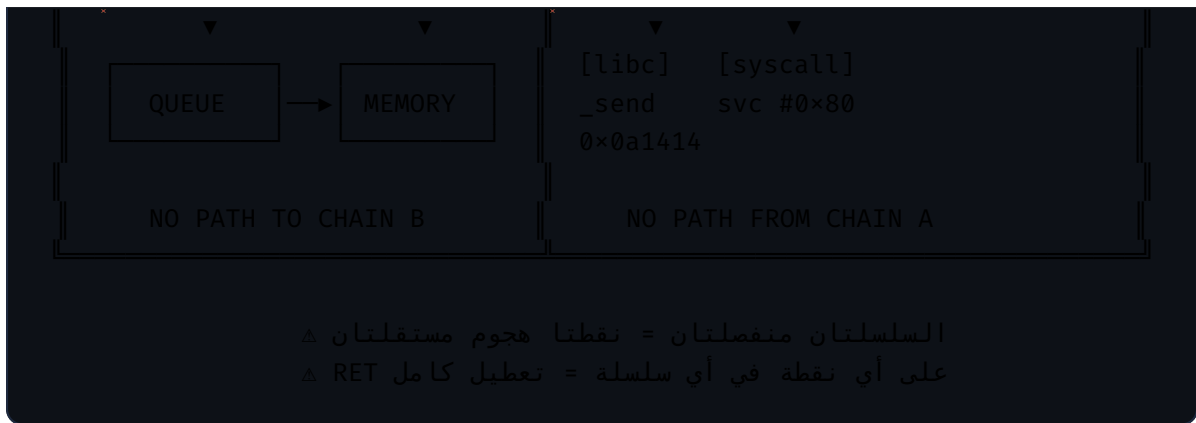
51

دالة كشف أمنية

١. المعمارية: سلسلتان منفصلتان

تحليل الـ call graph كشف حقيقة معمارية خطيرة: خط أنابيب الحماية ينقسم إلى **سلسلتين منفصلتين تماماً** — لا يوجد مسار استدعاء مباشر بينهما:





الخلاصة المعمارية الخطيرة

`report_emit` لا يصل أبداً إلى `transport_send` عبر الـ call graph. السلسلة A تجمع الأدلة وتضعها في الذاكرة (queue)، والسلسلة B ترسلها عبر الشبكة. هذا يعني أن RET على أي نقطة اختناق في أي سلسلة يكفي لتعطيل الحماية بالكامل.

٢. المستوى الأول — RET واحد = عمى 100% TIER 1 — كارثي

هذه الأوفستات هي أخطر ما يمكن أن يتعرض لـ RET hook. كل واحد منها نقطة فشل واحدة (SPOF) — تعطيلها يُعمي الحماية بالكامل بدون أي إنذار.

T1-1: 0x0373e8 — report_emit الأخطر

الخاصية	القيمة
النوع	دالة جمع الأدلة الوحيدة
Fan-in	5 (كل مسارات COREREPORT)
Fan-out	6 دوال
مسارات الكشف المعطلة	13/24 (54%)
الأثر	100% من أدلة COREREPORT تسقط

لماذا هي الأخطر؟

كل دالة كشف في النظام (memory scan, jailbreak check, hook detection, speed hack, إلخ) تستدعي report_emit لإرسال نتائجها. إذا رجعت فوراً (RET):

- الكشف يعمل — الدوال تفحص وتجد الغش
- لكن لا شيء يُبلغ — كل النتائج تختفي في الفراغ
- السيرفر يرى صمتاً طبيعياً — لأنه لا يعرف أن هناك شيء يجب الإبلاغ عنه

```
; ≡ report_emit entry point ≡ 0x0373e8: sub sp, sp, #0x50 ; <
0x0373ec: stp x24, x23, [sp, #0x10]
0x0373f0: ... 0x0373f4: cbz x0, #0x37454 ; silent drop #1
0x0373fc: cbz x20, #0x37454 ; silent drop #2 0x037420: cbz x0,
#0x37454 ; silent drop #3 0x037438: cbz x0, #0x37454 ; silent drop
#4 0x037454: ret ; < exit بدون log أو retry
```

مشكلة معمارية: 4 نقاط سقوط صامت

حتى بدون RET hook، الدالة نفسها فيها 4 تعليمات `cbz` تسقط البيانات بصمت. لا يوجد logging، لا retry، لا alert. هذا تصميم fail-open خطير.

طريقة الإصلاح

- **Prologue Integrity Check**: أضف فحص أول تعليمة عند كل استدعاء — إذا كانت `RET` بدلاً من `SUB SP`، أطلق إنذار
- **Canary Return Value**: اجعل الدالة تُرجع قيمة تأكيد (مثلاً magic number). المستدعي يتحقق — إذا لم يحصل عليها، يعرف أن الدالة مُعطّلة
- **Heartbeat Counter**: عدّاد يزيد مع كل استدعاء ناجح. إذا توقف العدّاد عن الزيادة رغم وجود أحداث كشف، هناك مشكلة
- **Redundant Path**: أنشئ مسار إبلاغ ثاني مستقل تماماً — لا يمر بنفس الدالة
- **Fail-Closed**: حوّل كل `cbz → silent drop` إلى `cbz → emergency report + flag`

T1-2: 0x05a654 — net_send_wrapper خطر

الخاصية	القيمة
النوع	غلاف إرسال الشبكة الوحيد
Fan-in	11 (كل مسارات الإرسال)
Fan-out	1 (<code>transport_send</code> فقط)
مسارات الكشف المعطّلة	12/24 (50%)
الأثر	لا شيء يغادر الجهاز

لماذا خطير؟

هذه الدالة هي **القمع الوحيد** لكل حركة البيانات نحو الشبكة في السلسلة B. كل الـ 11 مسار إرسال تمر من هنا. RET عند المدخل = البيانات تبقى في الذاكرة للأبد ولا تصل السيرفر أبداً.

```
; ≡ net_send_wrapper ≡ ; 11 callers → ALL converge here → only
calls transport_send 0x05a654: sub sp, sp, #0x30 ; ← RET هنا =
0x05a6xx: bl 0x0f8210 ; → transport_send ... الشبكة ميتة
لا يوجد مسار بديل - هذا هو القمع الوحيد ; ... (الاستدعاء الوحيد)
```

طريقة الإصلاح

- **Send Confirmation Protocol**: السيرفر يرسل ACK لكل حزمة. إذا لم يصل ACK خلال فترة = إنذار
- **Queue Overflow Monitor**: إذا امتلأ الـ queue ولم تُرسل بيانات = الإرسال معطل
- **Multiple Send Paths**: أنشئ مسار إرسال بديل مستقل (مثلاً عبر URLSession أو مكتبة مختلفة)
- **Self-Check at Entry**: فحص أول تعليمة للدالة من داخل المستدعي

خطير

T1-3: 0x0f8210 — transport_send

الخاصية	القيمة
النوع	بوابة النقل (syscall + libc)
Fan-in	1 (فقط net_send_wrapper)
Fan-out	1 (syscall_gateway أو _send)
مسارات الكشف المعطلة	12/24 (50%)
الأثر	كلا مساري النقل ميتان

لماذا خطر بشكل خاص؟

هذه الدالة تحتوي على المنطق الذي يقرر بين مسارين:

▪ **المسار الآمن:** `svc #0x80` (syscall مباشر — لا يمكن hook)

▪ **المسار العادي:** `_send` (libc — يمكن hook)

إذا تم RET عند مدخل الدالة، **كلا المسارين يموتان معاً**. لا يهم أن المسار الآمن موجود — لن يصل إليه أحد.

```
; == transport_send - dual-path selector == 0x0f8210: sub sp,
sp, #0x30 ; ← RET هنا = كلا المسارين ميتان 0x0f8230: ldr w8,
[x8, #0x430] ; read flag from 0x2ab430 0x0f8234: tbnz w8, #0xd,
#0xf824c ; bit13 → syscall path 0x0f8244: bl _send ; libc
(hookable!) ... 0x0f825c: bl sub_0a1414 ; syscall gateway (secure)
```

طريقة الإصلاح

▪ **Inline the Selector:** انقل منطق الاختيار (bit13 check) إلى كل مستدعي — لا

تعتمد على دالة واحدة

▪ **Prologue Guard:** قبل الاستدعاء، تحقق أن أول 4 بايت ليست `RET`

▪ **Fallback Direct Syscall:** إذا فشل الاستدعاء، نَقِّذ `svc #0x80` مباشرة من المستدعي

٣. المستوى الثاني — تأثير واسع — TIER 2 — خطير

T2-1: 0x0371e0 — channel_manager — خطير

الخاصية	القيمة
النوع	مدير قناة COREREPORT
Fan-in	5
مسارات الكشف المعطلة	13/24 (54%)
يتحكم بـ	kill-switch (0x29bc38) + channel lookup

الخطورة

هذه الدالة تقرأ `kill-switch byte` عند `0x29bc38` وتبحث عن قناة COREREPORT. إذا أرجعت RET فوراً:

- القناة لا تُجد أبداً — `report_emit` يحصل على NULL ويسقط البيانات بصمت
- ال `kill-switch` لا يُقرأ — لكن هذا لا يهم لأن القناة ميتة أصلاً

```
; == channel_manager - kill-switch reader == 0x0371e0: sub sp,
sp, #0x60 ... 0x037214: ldrb w0, [x8, #0xc38] ; read kill-switch
at 0x29bc38 0x037218: cbnz w0, #0x3724c ; != 0 -> channel alive
0x03721c: b #0x373cc ; = 0 -> SKIP everything ... 0x0373a8: ldr
w9, [x8, #0xc3c] ; retry counter at 0x29bc3c 0x0373b4: cmp w9, #3
; reached 3? 0x0373c4: strb wzr, [x8, #0xc38] ; KILL: write 0 ->
permanent disable
```

طريقة الإصلاح

- Kill-Switch Recovery:** أضف timer يعيد تشغيل القناة بعد فترة (بدلاً من الموت الدائم)
- Server-Side Heartbeat:** السيرفر يتوقع تقارير دورية — غيابها = إنذار

▪ **Redundant Channel Lookup**: استدعاء القناة من أكثر من مكان بمنطق مختلف

T2-2: 0x0376a0 — router (event dispatcher) **خطر**

الخاصية	القيمة
النوع	موزّع الأحداث عبر gate table
Fan-in	2 (عبر 0x0f1114, 0x0f10dc — vtable)
يتحكم بـ	gate table عند 0x29bc10 (7 slots)
الأثر	كل الأحداث من محرك اللعبة تسقط

الخطورة

الـ router يُستدعى من محرك اللعبة عبر vtable (لذلك fan-in = 0 في التحليل الثابت). يقرأ `event_id` ويبحث في جدول البوابات:

```
; ≡ router - gate table lookup ≡ 0x0376a0: sub sp, sp,
#0x50 ... 0x0376bc: cmp w20, #6 ; event_id > 6? 0x0376c0: b.hi
#0x376e4 ; → out of range, drop 0x0376c8: add x9, x9, #0xc10 ; x9
= 0x29bc10 (gate table) 0x0376cc: ldr w8, [x9, w20, uxtw #2] ;
slot[event_id] 0x0376d0: cbz w8, #0x376e4 ; slot=0 → silent drop
```

إذا أرجعت RET: كل الأحداث من محرك اللعبة (مثل teleport, speed hack detection, detection) لا تصل أبداً إلى `report_emit`.

طريقة الإصلاح

- **Direct Calls**: بدلاً من handler → router → vtable, اجعل بعض الكشوفات تستدعي `report_emit` مباشرة
- **Router Integrity**: فحص دوري أن أول تعليمة في الـ router لم تتغير
- **Multiple Dispatch Points**: لا تعتمد على router واحد — وزّع التوزيع

T2-3: `0x0f1114` / `0x0f10dc` — `dispatch entries` خطر

الخاصية	القيمة
النوع	نقاط دخول الأحداث (vtable callbacks)
Fan-in	0 (يُستدعيان عبر function pointer)
Fan-out	يُستدعيان <code>router</code> فقط
الأثر	محرك اللعبة يفقد الاتصال بالحماية

الخطورة

هاتان الدالتان هما **الجسر** بين محرك اللعبة ونظام الحماية. يُسجّلان ك `callbacks` عند التهيئة. `RET` على أي منهما = محرك اللعبة يستدعي الدالة لكنها ترجع فوراً بدون تمرير الحدث لـ `router`.

طريقة الإصلاح

- **Callback Verification**: تحقق دورياً أن الـ `callback` المسجّل لم يتغير أوله
- **Multiple Callbacks**: سجّل أكثر من `callback` لنفس الأحداث

T2-4: `0x0a1ff8` — `serializer_init` تأثير واسع

الخاصية	القيمة
النوع	مُهيّئ المُسلسل (protobuf-style)
Fan-in	161 (أكبر chokepoint في البايناري!)
مسارات الكشف عبره	22/24 (92%)
لكن...	155 مستدعي ليسوا anti-cheat — تعطيله يُعطّل البرنامج كله

الخطورة + التحفظ

هذه دالة بنية تحتية (infrastructure) — تستخدم `operator g` `pthread_once` `new` لتهيئة singleton المُسلسِل. 92% من مسارات الكشف تمر منها. لكن تعطيها لا يُعطل الحماية فقط — يُعطل **كل** الوظائف التي تستخدم السلسلة (serialization)، بما فيها وظائف اللعبة نفسها. هذا يجعل الكشف أسهل (اللعبة تتعطل).

طريقة الإصلاح

- **Lazy vs Eager Init**: هَيِّئ المُسلسِل مبكراً في الـ constructor قبل أي hook
- **Init Verification**: تحقق أن الـ singleton تم تهيئته بنجاح
- **Separate Serializer**: استخدم مُسلسِل منفصل لبيانات الحماية عن بيانات اللعبة

٤. المستوى الثالث — مستهدف لكن أضيق TIER 3 — محدود

T3-1: 0x0a1414 — syscall_gateway محدود

الخاصية	القيمة
النوع	بوابة syscall مباشر (svc #0x80)
Fan-in	9 (مشاركة: stat, read, open, recv, send, ...)
مسارات الكشف عنه	15/24 (63%)
الأثر	المسار الآمن يموت — لكن libc fallback يبقى

لماذا "محدود"؟

بوابة ال syscall تُستخدم عندما يكون bit13 في علم 0x2ab430 مفعلاً. إذا تم RET عليها:

- المسار الآمن (anti-hook) يموت
 - لكن transport_send يسقط إلى _send (libc) — البيانات لا تزال تُرسل
 - المشكلة: مسار libc يمكن hook بسهولة — لكنه يحتاج hook إضافي
- لذلك هذه ليست SPOF حقيقية — تحتاج هجوم مزدوج (RET + libc hook).

```
; ≡ syscall_gateway ≡ 0x0a1414: ldp x1, x2, [sp] ; load
syscall args 0x0a1418: ldp x3, x4, [sp, #0x10] 0x0a141c: ldp x5,
x6, [sp, #0x20] 0x0a1424: mov x16, #0 ; syscall number in x16
0x0a1428: svc #0x80 ; ← kernel trap 0x0a142c: b.lo #0xa1444 ;
success → return
```

طريقة الإصلاح

▪ **Inline Syscall:** نفذ svc #0x80 مباشرة في كل مستدعي (لا تعتمد على دالة واحدة)

▪ **Gateway Integrity:** تحقق دورياً أن أول تعليمة لم تصبح RET

▪ **Detect Fallback**: إذا سقط النظام إلى libc دائماً رغم تفعيل bit13 — أرسل إنذار

T3-2: 0x035348 — config_parser (CSSCRIPT/TSS) — محدود

الخاصية	القيمة
النوع	محلل إعدادات القناة
Fan-in	1 (channel_manager فقط)
الأثر	إعدادات القناة يفشل → القناة لا تُهيأ

هذه الدالة تحلل نص الإعدادات (يحتوي "TSS"، "CSSCRIPT"، إصدار "7.7.31.5614"). RET عليها = إعدادات القناة يفشل بصمت → نفس أثر تعطيل channel_manager لكن بطريقة غير مباشرة.

طريقة الإصلاح

- **Config Validation**: تحقق من نتيجة التحليل — إذا فشل، لا تستمر بصمت
- **Hardcoded Fallback**: إذا فشل التحليل، استخدم إعدادات افتراضي مُسقّر

T3-3: 0x0370d0 — timestamp_util — ثانوي

الخاصية	القيمة
النوع	دالة الطابع الزمني
Fan-in	5 (كل دوال الأنبوب تستدعيها)
تستخدم	time (libc)
الأثر	بيانات التوقيت تفسد — لكن الأنبوب قد يستمر

RET هنا يُرجع قيمة عشوائية (ما كان في x_0) بدلاً من الوقت. التقارير قد تستمر لكن بتوقيت فاسد — السيرفر قد يرفضها أو قد لا يلاحظ. هذه ثغرة أقل خطورة.

٥. جدول الملخص الشامل

المستوى	الأوفست	الاسم	-Fan in	مسارات كشف معطلة	الأثر	يُكشف؟
T1	0x0373e8	report_emit	5	13/24 (54%)	100% أدلة COREREPORT	لا
T1	0x05a654	net_send_wrapper	11	12/24 (50%)	100% إرسال شبكة	لا
T1	0x0f8210	transport_send	1	12/24 (50%)	syscall + libc ميتان	لا
T2	0x0371e0	channel_manager	5	13/24 (54%)	قناة COREREPORT ميتة	لا
T2	0x0376a0	router	*2	أحداث المحرك	أحداث اللعبة تسقط	لا
T2	0x0f10dc	dispatch_entry_A	vtable	أحداث المحرك	callback ميت	لا
T2	0x0f1114	dispatch_entry_B	vtable	أحداث المحرك	callback ميت	لا
T2	0x0a1ff8	serializer_init	161	22/24 (92%)	كل serialization	نعم (crash)
T3	0x0a1414	syscall_gateway	9	15/24 (63%)	مسار آمن فقط	لا
T3	0x035348	config_parser	1	غير مباشر	إعداد القناة يفشل	لا
T3	0x0370d0	timestamp_util	5	—	توقيت فاسد	ممكن

* router يُستدعى عبر vtable/function pointer — لذلك الـ 2 static fan-in لكن الـ dynamic fan-in أعلى بكثير

٦. لماذا لا يُكشف المهاجم؟

تحليل البايनاري كشف عن غياب تام لأي آلية حماية ذاتية:

آلية الحماية	الحالة	الدليل
Code Integrity Check	غير موجود	لا يوجد <code>vm_read</code> أو CRC/hash على <code>TEXT. 7_</code> استدعاءات <code>vm_protect</code> كلها لـ Memory Integrity detection (تفحص ذاكرة اللعبة)، لا لحماية ذاتية
/ Heartbeat Keep-Alive	غير موجود	لا يوجد "heartbeat" string أو "keepalive" أو "ping" في البايناري. السيرفر لا يتوقع إشارات حياة
Watchdog Thread	غير موجود	لا يوجد thread مراقب يتحقق من حالة الأنبوب
Function Prologue Check	غير موجود	لا دالة تقرأ أول تعليمة في دالة أخرى للتحقق من RET hook
Anti-Hook on Self	غير موجود	الـ hook detection يفحص دوال اللعبة والنظام — لا يفحص دوال الحماية نفسها
Fail-Closed Design	غير موجود	كل نقاط الفشل تُسقط البيانات بصمت (cbz → ret) بدلاً من إطلاق إنذار

الاستنتاج الأمني

النظام مصمم على نمط **Fail-Open** — إذا فشل أي شيء، يصمت بدلاً من أن يُنذر. هذا يعني أن أي RET hook على أي نقطة اختناق سيُنتج **صمتاً مطابقاً تماماً** للحالة الطبيعية من منظور السيرفر. لا يمكن للسيرفر التمييز بين "لا يوجد غش" و "الحماية مُعطّلة".

٧. خطة الإصلاح الشاملة

١. تحويل كل SPOF إلى تصميم Fail-Closed

- كل `cbz → silent ret` يجب تحويله إلى `cbz → emergency_report + flag`
- كل فشل يجب أن يُنتج إشارة وليس صمت

٢. إضافة Server-Side Heartbeat

- السيرفر يتوقع تقريراً دورياً كل N ثانية ("أنا حي والحماية تعمل")
- غياب التقرير = إنذار فوري (الحماية مُعطلة أو الاتصال مقطوع)
- هذا يكشف كل أنواع التعطيل — RET hooks, memory patches, إلخ

٣. Code Integrity Self-Check

- احسب hash لأوائل كل دالة حرجة عند التهيئة
- تحقق دورياً أن الـ hash لم يتغير
- إذا تغيّر = الدالة مُعدّلة (RET hook أو NOP patch)

٤. كسر نقاط الاختناق (SPOF Elimination)

- أنشئ مسار إبلاغ ثاني مستقل تماماً عن `report_emit`
- استخدم مُسلسل منفصل لبيانات الحماية
- أنشئ قناة نقل بديلة لا تمر بـ `transport_send`
- الهدف: لا يمكن لـ RET hook واحد أن يُعمي كل شيء

٥. Anti-Tamper على الدوال الحرجة

- سجّل أول 4/8 بايت من كل دالة حرجة عند بدء التشغيل
- في thread منفصل، تحقق كل ثانية أنها لم تتغير
- إذا أصبحت `0xD65F03C0` (RET instruction) hook = مكشوف

٦. ربط السلسلتين

- Chain A (جمع الأدلة) و Chain B (إرسال الشبكة) يجب أن تتبادلا إشارات صحة
- إذا كانت Chain A تجمع لكن Chain B لا ترسل = إنذار
- إذا كانت Chain B ترسل لكن Chain A لا تجمع = إنذار

ANOGS/AnoSDK RET Hook Vulnerability Analysis — Defensive Report

SHA-256: 0a9943f19c78194e377210dce5b349cf13b4e544dda63668cb39239381b159c1

Binary: 2,871,200 bytes | 10,121 functions | 289 imports | 51 security functions

✦ bypass by Hdanh — made by @Nopermcc ✦